

Data Mining Practice 12_최종

Team 2

양승렬 2200409

최서현 22100133

문벼리 22000245

[Baseline model](#)

[Task1](#)

[Task2](#)

[Task3](#)

[Task4](#)

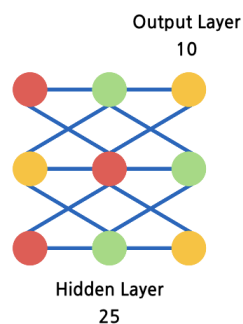
[Bonus 1](#)

keras 패키지를 사용하여 아래 task를 수행하여라.

(R뿐만 아니라 python의 keras 패키지를 사용하는 것도 가능하다.)

강의자료에서 modeling 한 neural net architecture를 적용해보고 그 외에도 다양한 architecture를 사용하여 성능을 평가하여 보고, 비교하여 보아라.

Baseline model



- 수업 중 배운 architecture와 동일한 Baseline 모델 만들기

[환경 설정]

```
### 라이브러리 불러오기
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

```

from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import random
import matplotlib.pyplot as plt

### 시드 값 고정
seed = 2024
np.random.seed(seed)
tf.random.set_seed(seed)
random.seed(seed)

```

[데이터 불러오기]

```

### 데이터 불러오기
(x_train, y_train), (x_test, y_test) = mnist.load_data()

### shape 확인
print(f"x_train shape: {x_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"x_test shape: {x_test.shape}")
print(f"y_test shape: {y_test.shape}")

```

```

x_train shape: (60000, 28, 28)
y_train shape: (60000,)
x_test shape: (10000, 28, 28)
y_test shape: (10000,)

```

- 60000개의 이미지가 28x28 픽셀의 2차원 배열로 이루어져 있음
- 총 784개의 픽셀 값 (0~225 사이의 숫자)

[전처리]

```

### 정규화
x_train = x_train / 255.0
x_test = x_test / 255.0

```

- 0 ~ 225 사이의 범위를 가지는 픽셀값을 0과 1 사이로 스케일링

```

### 목적변수 one-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

```

- 현재 종속변수는 숫자 0~9로 표현되어 있다.

- 사람은 이를 클래스 레이블, 즉 카테고리로 이해할 수 있지만, 모델은 단순히 숫자(연속값)로 인식할 가능성이 있다.
- 이를 방지하기 위해, One-hot encoding을 사용해 숫자를 벡터 형태로 변환한다. 이를 통해 모델이 각 숫자를 독립된 클래스로 정확히 학습할 수 있다.

[모델링]

```
#### node(units) 수 설정
input_nodes = x_train.shape[1] * x_train.shape[2]
hidden_nodes = 25
output_nodes = 10
```

- PPT와 동일한 구조의 architecture 사용

[모델 설정]

```
#### 모델 설정
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(hidden_nodes, activation='sigmoid'),
    Dense(output_nodes, activation='softmax')
])

#### 모델 요약 출력
model.summary()
```

- Flatten : 현재 MNIST 데이터의 한 이미지(28x28 크기)는 2D 형태이다.
 - Dense 층은 1차원 벡터만 처리할 수 있기 때문에 이를 784 크기의 1차원 벡터로 변경해야 한다.
 - Hidden Layer의 activation function = `sigmoid`
 - Output Layeractivation function = `softmax`
- ▼ `softmax`

2. Softmax

: 로지스틱 회귀분석을 일반화

$$\text{Logistic} \quad \begin{cases} \times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \\ \circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \end{cases}$$

→ Softmax로 일반화

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1|\vec{x})$$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2|\vec{x})$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3|\vec{x})$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4|\vec{x})$$

Softmax regression

$$y = 1, 2, 3, \dots, N$$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j$$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

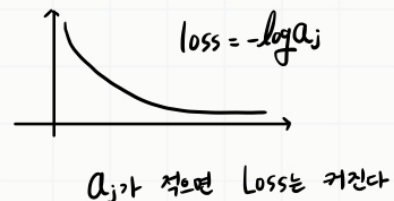
- Cost function

① logistic

$$\text{loss} = -y \log a_1 - (1-y) \log a_2 \quad \rangle \quad J(\vec{w}, b) = \text{average loss}$$

② Softmax

$$\text{loss}(a_1, \dots, a_N, y) = \begin{cases} -\log a_1 \\ -\log a_2 \\ \vdots \\ -\log a_N \end{cases}$$



[모델 컴파일]

```

### 모델 컴파일
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

```

[모델 학습]

```

### 모델 학습
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.1, # validation 10%
    verbose=1
)

```

```

Model: "sequential"

```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 25)	19625
dense_1 (Dense)	(None, 10)	260

```

=====
Total params: 19885 (77.68 KB)
Trainable params: 19885 (77.68 KB)
Non-trainable params: 0 (0.00 Byte)

```

- 첫 번째 layer의 theta = $785 * 25 = 19,624$
- 두 번째 layer의 theta = $26 * 10 = 260$

[모델 평가]

```

### 모델 평가
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print(f"Train Accuracy: {train_acc:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

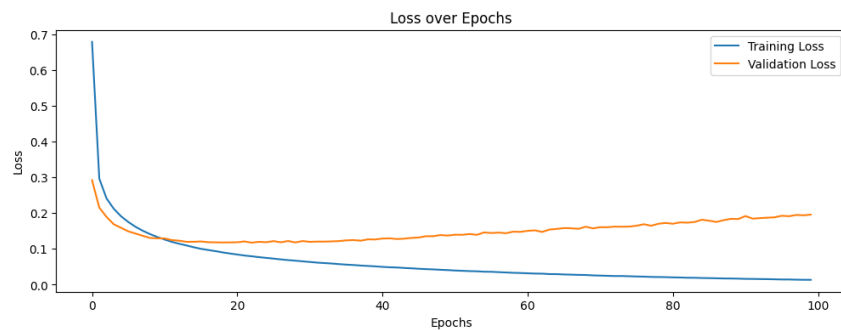
```

모델 이름	Baseline
-------	----------

train accuracy	0.9654
test accuracy	0.9527

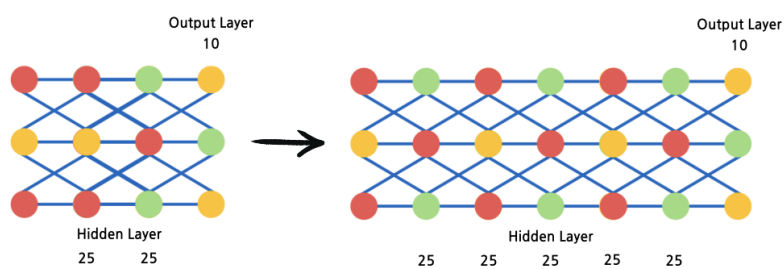
[학습 결과 시각화]

```
# Loss
plt.figure(figsize=(12, 4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



Task1

- Layer 수를 변경



- Hidden Layer의 수를 2 ~ 5까지 늘려가며 확인

[모델 학습 함수 만들기]

```
### 모델 학습 함수 만들기

def build_and_evaluate(hidden_layers):
    # Layer node 수 설정
```

```

input_nodes = x_train.shape[1] * x_train.shape[2]
output_nodes = 10
hidden_nodes = 25

# 모델 생성
model = Sequential([Flatten(input_shape=(28, 28))])
# Layer 수에 따라 Dense 추가 (activation function = sigmoid)
for _ in range(hidden_layers):
    model.add(Dense(hidden_nodes, activation='sigmoid'))
model.add(Dense(output_nodes, activation='softmax'))

# 모델 컴파일
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# 모델 학습
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.1, # validation 10%
    verbose=0
)

# 모델 평가
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print(f"Hidden Layers: {hidden_layers} | Train Accuracy: {train_acc:.4f}
      | Test Accuracy: {test_acc:.4f}")
return train_acc, test_acc, history

```

[모델 평가]

```

### 모델 평가

# hidden layer 수를 2~5까지 바꿔가며 train, test accuracy 확인
results = {}
for hidden_layers in range(2, 6):
    train_acc, test_acc, history = build_and_evaluate(hidden_layers)
    results[hidden_layers] = {
        "train_acc": train_acc,
        "test_acc": test_acc,

```

```
"history": history
}
```

layer 수	train accuracy	test accuracy
2	0.9706	0.9558
3	0.9684	0.9534
4	0.9628	0.9464
5	0.9487	0.9316

- 예상 : layer 수가 증가할수록 모델이 복잡해져 train의 성능은 좋아지지만 test에서는 성능 감소
- 결과 : layer 수가 2일 때까지는 accuracy가 증가했지만 그 이후는 오히려 train에서도 성능이 감소되는 것을 확인 (과적합은 예상대로 더욱 심해짐)

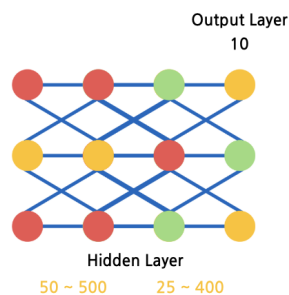
[최적 layer 판단]

- layer가 2일 때 train, test 모두 성능이 baseline보다 높기 때문에 layer가 2일 경우 최적이라 판단

모델 이름	Baseline	Task1
train accuracy	0.9654	0.9706
test accuracy	0.9527	0.9558

Task2

- Layer당 unit의 수를 변경



[모델 학습 함수 만들기]

```
### 함수 정의
### Layer 노드 수 조합별 모델 생성 및 평가

def build_and_evaluate(node1, node2):
    model = Sequential([
        Flatten(input_shape=(28, 28)),
        # 다양한 node를 시도하기 위해 변수로 설정
        Dense(node1, activation='sigmoid'),
        Dense(node2, activation='sigmoid'),
        Dense(10, activation='softmax')
```

```

])

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# 모델 학습
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.1, # validation 10%
    verbose=0
)

# 모델 평가
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
return train_loss, train_acc, test_loss, test_acc, history

```

[다양한 node 실험]

```

### 다양한 노드 수 조합 테스트
node_combinations = [(50, 25), (100, 50), (200, 100),
                     (300, 200), (400, 300), (500, 400)]
results = {}

for (node1, node2) in node_combinations:
    train_loss, train_acc, test_loss,
    test_acc, history = build_and_evaluate(node1, node2)
    results[(node1, node2)] = {
        "train_loss": train_loss,
        "train_acc": train_acc,
        "test_loss": test_loss,
        "test_acc": test_acc,
        "history": history
    }
print(f"Node1: {node1}, Node2: {node2}
      | Train Acc: {train_acc:.4f}, Test Acc: {test_acc:.4f}")

```

node 조합	train accuracy	test accuracy
(50, 25)	0.9829	0.9698
(100, 50)	0.9903	0.9736
(200, 100)	0.9952	0.9798

(300, 200)	0.9939	0.9785
(400, 300)	0.9942	0.9767

- 예상 : node 수가 증가할 수록 모델의 성능이 높아지지만 너무 많은 node는 모델 복잡도를 증가시켜 과적합 유발
- 결과 : node 수가 증가할 수록 모델의 성능이 높아지는 것을 확인 더욱 많은 node를 실험해보고 싶었지만 학습 시간의 증가로 인해 (500, 400)까지 학습

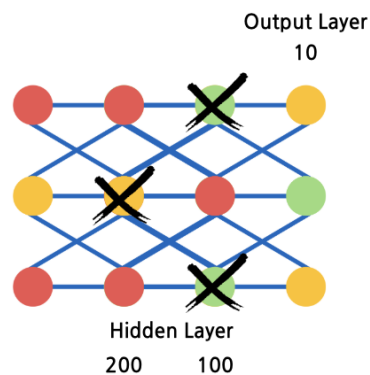
[최적 모델 선정]

- 물론 노드 수를 늘릴수록 train, test 모두 성능이 향상하는 것을 확인했지만, 학습 시간을 고려해 accuracy가 급격하게 증가한 (200, 100)을 최적의 모델로 선정

모델 이름	Baseline	Task1	Task2
train accuracy	0.9654	0.9941	0.9952
test accuracy	0.9527	0.9580	0.9798

Task3

- Drop-out을 추가하고 drop-out rate를 변경



[모델 학습 함수 만들기]

모델 학습 함수 만들기

```
def build_and_evaluate(dropout_rate):
    model = Sequential([
        Flatten(input_shape=(28, 28)),
        Dense(200, activation='relu'),
        Dropout(dropout_rate), # 다양한 Dropout 조합 실험을 위해 변수 추가
        Dense(100, activation='relu'),
        Dropout(dropout_rate),
        Dense(10, activation='softmax')
    ])
```

```

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# 모델 학습
history = model.fit(
    x_train, y_train,
    epochs=100,
    validation_split=0.1,
    verbose=0
)

# 모델 평가
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
return train_loss, train_acc, test_loss, test_acc, history

```

[다양한 Dropout 비율 설정]

```

### 다양한 Dropout 비율 조합 테스트
dropout_rates = [0.1, 0.2, 0.4, 0.5, 0.6]
results = {}

for rate in dropout_rates:
    train_loss, train_acc, test_loss, test_acc,
    history = build_and_evaluate(rate)
    results[rate] = {
        "train_loss": train_loss,
        "train_acc": train_acc,
        "test_loss": test_loss,
        "test_acc": test_acc,
        "history": history
    }
print(f"Dropout Rate: {rate:.1f} | Train Acc: {train_acc:.4f},
      Test Acc: {test_acc:.4f}")

```

Dropout 비율	train accuracy	test accuracy
0.1	0.9936	0.9782
0.2	0.9919	0.9805
0.4	0.9882	0.9780
0.5	0.9848	0.9748
0.6	0.9800	0.9710

[최적 모델 선정]

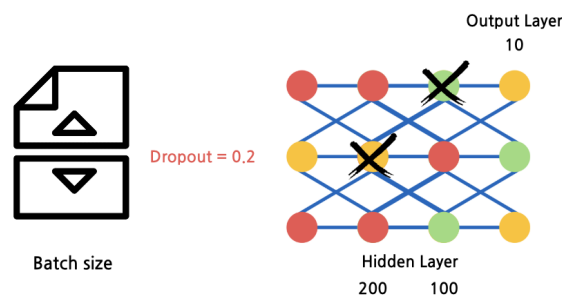
- train, test 모두 성능이 오른 경우인 Dropout = 0.2 (20% 제거)를 최적 모델로 선정

모델 이름	Baseline	Task1	Task2	Task3
train accuracy	0.9654	0.9941	0.9952	0.9919
test accuracy	0.9527	0.9580	0.9798	0.9805

- 이전보다 과적합이 줄어든 것을 확인

Task4

- Batch size 변경
- batch는 전체 train sample을 한번에 gradient에 합산하지 않고, sample의 일부씩 (batch size만큼) gradient를 계산해서 업데이트를 하는 방법이다. 다양한 Batch size를 시도해보고 어떤 변화가 있었는지 확인해보라



- Mini-Batch

1. Mini-batch Gradient Descent

- Vectorization을 통해 우리는 효율적으로 전체 데이터 (m)에 대한 값을 빠르게 계산할 수 있다. 비효율적!

→ 그러나 m이 5,000,000개 라면? → 경사하강법 한번 업데이트 (필름) 할 때마다 5만 번씩 반복

해결방법: Baby training set을 만든다 (mini-batch)

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(1000)} & x^{(1001)} & \dots & x^{(2000)} & \dots & x^{(5,000,000)} \\ x^{(1)}_{13} & x^{(2)}_{13} & \dots & x^{(1000)}_{13} & x^{(1001)}_{13} & \dots & x^{(2000)}_{13} & \dots & x^{(5,000,000)}_{13} \end{bmatrix}$$

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(1000)} & y^{(1001)} & \dots & y^{(2000)} & \dots & y^{(5,000,000)} \\ y^{(1)}_{13} & y^{(2)}_{13} & \dots & y^{(1000)}_{13} & y^{(1001)}_{13} & \dots & y^{(2000)}_{13} & \dots & y^{(5,000,000)}_{13} \end{bmatrix}$$

- Mini-batch t: $X^{t:t3}, Y^{t:t3}$

+ 헛갈리는 표기법 정리: $x^{(i)}$ = i번째 데이터 셋 $z^{[1]}$ = 1번째 레이어

$x^{t:t3}$ = t번째 mini-batch

- Mini-batch gradient descent

```

for t = 1, ..., 5000 {
  Forward Prop on  $X^{t:t3}$ 
   $Z^{t3} = W^{t3}X^{t:t3} + b^{t3}$ 
   $A^{t3} = g^{t3}(Z^{t3})$ 
   $\vdots$ 
   $A^{t3} = g^{t3}(Z^{t3})$ 
  Compute Cost  $J^{t:t3} = \frac{1}{1000} \sum_{i=1}^I \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_i \|W^{t3}\|_F^2$ 
  Backprop to Compute gradient  $J^{t:t3}$  (Using  $X^{t:t3}, Y^{t:t3}$ )
   $W^{t3} := W^{t3} - \alpha dw^{t3}, b^{t3} := b^{t3} - \alpha db^{t3}$ 
}

```

Vectorized

[모델 학습 함수 만들기]

모델 학습 함수 만들기

```

def build_and_evaluate(batch_size):
    model = Sequential([
        Flatten(input_shape=(28, 28)),
        Dense(200, activation='sigmoid'),
        Dropout(0.2), # Dropout = 0.2
        Dense(100, activation='sigmoid'),
        Dropout(0.2),
        Dense(10, activation='softmax')
    ])

    model.compile(
        loss='categorical_crossentropy',
        optimizer='adam',
        metrics=['accuracy']
    )

    # 모델 학습
    history = model.fit(
        x_train, y_train,
        epochs=10,
        # 다양한 batch_size 실험을 위해 변수로 설정
        batch_size=batch_size,
        validation_split=0.1,
        verbose=0
    )

    # 모델 평가
    train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)

```

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
return train_loss, train_acc, test_loss, test_acc, history
```

[다양한 Dropout 비율 설정]

```
### 다양한 Dropout 비율 조합 테스트
batch_sizes = [8, 16, 32, 64, 128, 256, 512, 1024, 2048]

for batch_size in batch_sizes:
    train_loss, train_acc, test_loss,
    test_acc, history = build_and_evaluate(batch_size)
    results[batch_size] = {
        "train_loss": train_loss,
        "train_acc": train_acc,
        "test_loss": test_loss,
        "test_acc": test_acc,
        "history": history
    }
    print(f"Batch Size: {batch_size} | Train Acc: {train_acc:.4f},
          Test Acc: {test_acc:.4f}")
```

Batch Size	train accuracy	test accuracy
16	0.9923	0.9801
32	0.9907	0.9771
64	0.9854	0.9741
128	0.9810	0.9722
256	0.9695	0.9627
512	0.9556	0.9519

- 60,000개의 데이터를 batch size 64로 나누면 총 937개의 mini-batch가 만들어지고, 마지막 mini-batch에는 32개의 남은 샘플이 포함된다.

[추가 실험]

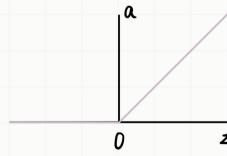
- activation function = **ReLU**

▼ **ReLU**

② ReLU

: Rectified Linear Unit

$$\text{ReLU}(x) = \max(0, x) \rightarrow$$



- ReLU의 장점

1) 계산 효율성

: 지수 함수가 포함된 Sigmoid보다 ReLU가 계산 효율성이 높다.

2) Gradient (기울기) 소실 방지

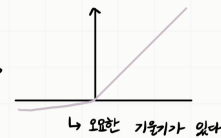
: ReLU는 양쪽 입력값에 대해 일정한 기울기를 가지고 있다.

+ Saturation problem 방지
Sparse activation 생성

→ 0 이하일 때는 아예 기울기가 없다?

→ z가 0.000이 나오는 경우는 거의 없다.

→ Leaky ReLU 라는 애도 있다



Batch Size	train accuracy	test accuracy
16	0.9914	0.9799
32	0.9916	0.9790
64	0.9934	0.9800
128	0.9935	0.9805
256	0.9919	0.9797
512	0.9896	0.9797

[최적 모델 선정]

- 이전 모델보다 test에서는 성능의 향상을 보이지 않음
- 그나마 ReLU Batch Size = 128에서 train 성능 향상을 확인

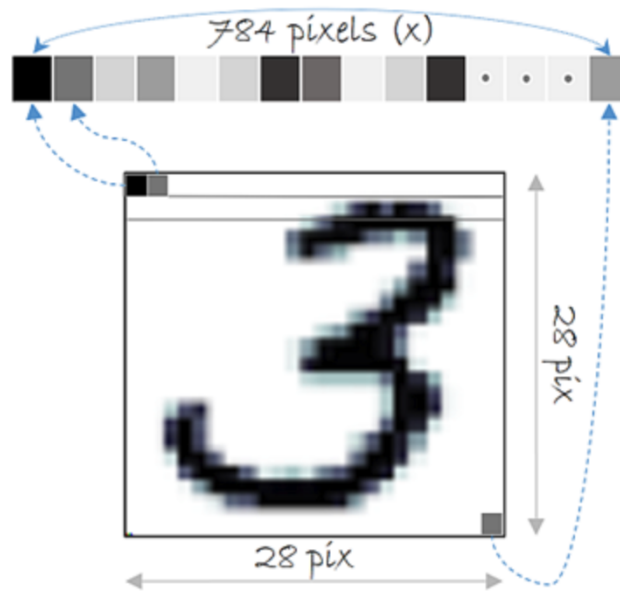
모델 이름	Baseline	Task1	Task2	Task3	Task4
train accuracy	0.9654	0.9941	0.9952	0.9919	0.9935
test accuracy	0.9527	0.9580	0.9798	0.9805	0.9805

- 그러나 Task4의 작업은 큰 의미 없음

Bonus 1

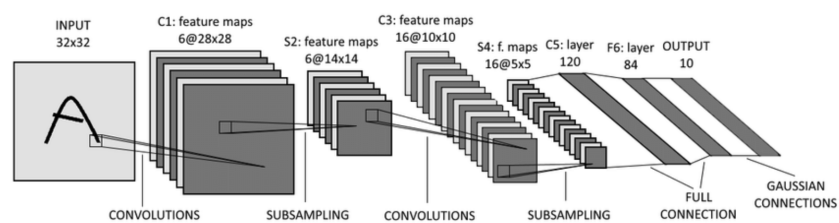
- CNN 모델 사용

CNN 배경은 overfitting으로 부터 시작된다. Neural Network를 깊게 만들수록 과적합의 가능성이 높다. 그 이유는 하나의 사진을 fully connected layer에 집어넣기 위해 직렬화를 수행하면 상관관계를 잃어버리는 문제점이 생긴다.

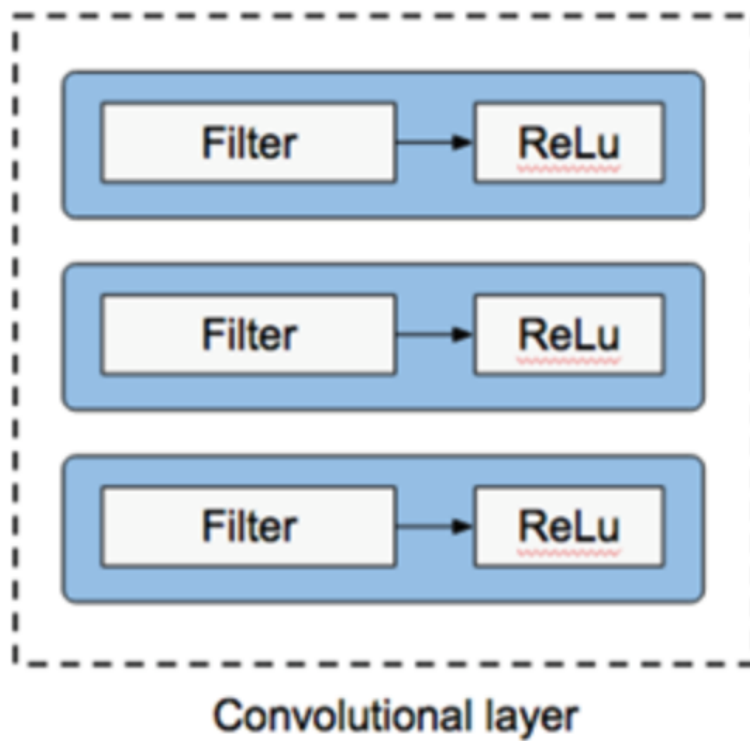


CNN은 이러한 문제점을 해결하기 위해 fully connected layer 앞 부분에 convolution layer와 pooling layer를 추가했다.

[CNN 구조]



[convolution layer]



- convolution layer는 입력 데이터로부터 특징을 추출하는 역할이다.
- filter를 통해 이미지의 특징을 찾고 일정 간격으로 원본 이미지에서 이동시키면서 연산을 적용합니다.
- 이를 통해 얻어낸 결과를 feature map이다.

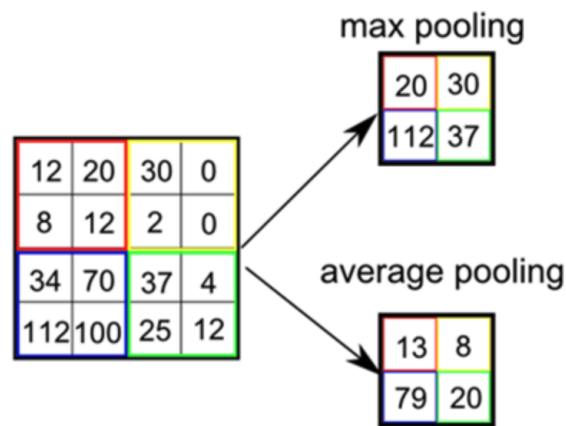
즉, CNN의 가장 큰 아이디어는 이미지 전체가 아닌 특징을 보는 것!

Pooling

pooling: 추출된 activation map을 인위로 줄이는 이 작업이다.

기법

- average
- max
- min pooling



Max pooling은 특징의 값이 가장 큰 값을 대표

average pooling은 평균값으로 대표

즉, 특징 중에서 가장 강한 신호 또는 중요한 신호를 더 압축시켜 전달하고 나머지는 무시하여 계산량을 줄이고 일 반화를 시킬 수 있다.

최종적으로 위 방식을 반복하여 feature extraction을 진행하고 Flatten() 함수를 통해 2차원 데이터를 1차원 데이터로 변환시켜 판단을 하는 것이다.

[CODE]

```
# 1. 데이터 불러오기 및 전처리
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0
y_train = to_categorical(y_train, 10) # One-hot encoding
y_test = to_categorical(y_test, 10)
```

```
# 2. 모델 구성 (CNN)
model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.2),
    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(10, activation='softmax')
])
```

```
# 3. 모델 컴파일
model.compile(
    loss='categorical_crossentropy', # 다중 클래스 분류를 위한 손실 함수
    optimizer='adam',               # 최적화 알고리즘
    metrics=['accuracy']             # 평가 지표: 정확도
)
```

```
# 4. 모델 학습
history = model.fit(
    x_train, y_train,
    epochs=20,                # Epoch 수
    batch_size=64,            # Batch Size
    validation_split=0.1,      # 검증 데이터 비율
    verbose=1                  # 학습 로그 출력
)
```

```
# 5. 모델 평가
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print(f"Train Accuracy: {train_acc:.4f}, Test Accuracy: {test_acc:.4f}")
```

Train Accuracy: 0.9989

Test Accuracy: 0.9931